

CI/CD пайплайны

Правдин Иван

24.05.2025

Наш webhook сервер работает:

- Git push → автоматический деплой
- Разные ветки обновляют разные окружения

Но:

- Изменения вебхук сервера требуют ручного вмешательства.
- Нет простого доступа к логам скриптов, нет статуса деплоя.
- Требуются вычислительные мощности под работу вебхук сервера.
-

В реальных проектах используют готовые платформы:

- **Jenkins** — самый популярный, self hosted
- **GitHub Actions** — встроен в GitHub, есть бесплатный режим
- **GitLab CI** — встроен в GitLab, есть self hosted режим
- **Azure DevOps, TeamCity, CircleCI...**

Общая идея: вместо своего webhook сервера используем платформу (отдельное приложение), которая позволяет выполнять скрипты по событиям в Git репозитории.

Что дают готовые платформы:

- **Надежность** — отказоустойчивость, бэкапы
- **Масштабируемость** — одна платформа используется под несколько проектов
- **Удобство** — веб-интерфейс, история запусков, логи
- **Безопасность** — управление паролями и ключами
- **Интеграции** — уведомления, мониторинг, тесты
- **Поддержка** — обновления, документация, сообщество

Компромисс: меньше контроля, но намного больше возможностей

Pipeline — последовательность шагов для доставки кода

Типичные шаги:

1. Получить код из Git
2. Собрать приложение (если нужно)
3. Запустить тесты
4. Задеплоить на сервер
5. Проверить что все работает

Наш webhook делал примерно то же самое, но:

- Все в одном скрипте
- Нельзя легко изменить последовательность
- Сложно добавить новые шаги

Выбираем GitHub Actions для демо:

- Код уже лежит в GitHub
- Не нужно поднимать отдельные сервера
- Хорошая интеграция с репозиторием
- Много готовых компонентов в marketplace

Базовые понятия:

- **Workflow** — описание pipeline в YAML файле
- **Job** — группа связанных шагов
- **Step** — отдельная команда или действие
- **Action** — готовый переиспользуемый компонент

Демо: Переносим webhook на GitHub Actions

Задача: воспроизвести логику нашего webhook сервера

Что будем делать:

- Создаем файл `.github/workflows/deploy.yml`
- Настраиваем срабатывание на push в main
- Добавляем шаги: checkout кода и деплой
- Используем SSH для подключения к серверу
- Тестируем и смотрим результат

Покажем на реальном репозитории с нашим веб-приложением

Сравнение с webhook сервером: Было (webhook)

- Свой сервер на VM
- Логи в консоли
- Один проект = один скрипт
- Нужно следить за работой

Стало (GitHub Actions)

- Управляемая инфраструктура
- Веб-интерфейс с историей
- Один файл конфигурации
- Работает автоматически

Функциональность та же, но качество решения выше

Наш pipeline пока только деплоит. А если добавить тесты?

Continuous Integration — это:

- Автоматический запуск тестов на каждый commit
- Проверка качества кода (линтеры, формatters)
- Быстрая обратная связь разработчику
- Предотвращение попадания багов в продакшн

Практически: тесты падают → деплой блокируется

Результат: в production попадает только рабочий код

Continuous Deployment — автоматизация доставки:

- Автоматический деплой после успешных тестов
- Управление разными окружениями (dev, staging, prod)
- Возможность быстрого отката при проблемах
- Мониторинг состояния после деплоя

Наш webhook уже делал CD, но примитивно

CI/CD вместе = полная автоматизация от commit до production

Куда двигаться дальше с нашим pipeline:

- **Добавить тесты** — unit, integration, e2e
- **Проверки кода** — ESLint, Prettier, security scans
- **Множественные окружения** — staging перед production
- **Уведомления** — Slack, email при проблемах
- **Мониторинг** — проверка здоровья после деплоя

Принцип: начинаем просто, постепенно усложняем

- **Fail fast** — останавливаемся при первой ошибке
- **Быстрая обратная связь** — pipeline должен работать быстро
- **Версионирование конфигурации** — .yaml файлы в Git
- **Переиспользование** — используем готовые Action'ы
- **Безопасность** — никаких паролей в коде, только secrets

Главное: автоматизация должна упрощать жизнь, а не усложнять

- Pipeline — это эволюция нашего webhook сервера
- GitHub Actions решает проблемы самодельных решений
- CI = автоматические тесты, CD = автоматический деплой
- Начинаем с простого деплоя, постепенно добавляем проверки

Следующий шаг: попробовать добавить тесты в свой pipeline

Вопросы?